

Conception et réalisation d'un pipeline Big Data de traitement de logs

Kafka → Spark (Streaming + Batch) → HDFS → Exports CSV → Power BI

Réalisé par : Abdelouahed ABBAD

Encadré par : M. HASSAN BADIR

Groupe : GINF3

Table des matières

1. Introduction.....	4
Objectifs pédagogiques et techniques :	4
Périmètre du travail :.....	4
2. Contexte et cas d'usage (logs applicatifs)	5
Exemple de structure d'un log (JSON) :.....	6
3. Architecture générale et composants	7
3.1 Vue d'ensemble des étapes	7
3.2 Conteneurs et rôles	7
4. Déploiement avec Docker Compose	9
4.1 Configuration Kafka	9
4.2 Configuration HDFS (NameNode/DataNode).....	9
4.3 Configuration Spark (master/worker)	9
5. Ingestion : génération de logs et publication dans Kafka	10
5.1 Producteur Node.js (producer.js).....	10
Exemples de variations utiles :	10
Commandes (exemple) :	10
6. Traitement temps réel : Spark Structured Streaming	11
6.1 Lecture depuis Kafka	11
6.2 Transformation et enrichissement	11
6.3 Écriture dans HDFS	11
Exemples de chemins HDFS :	11
Démarrage du streaming (exécution interactive) :	11
Arrêt propre :	11
7. Analytique batch : Spark SQL	12
7.1 Mise en place de la base logique (logsdb).....	12
7.2 Indicateurs calculés	12
Les KPI retenus pour le projet :	12
Exécution du batch :	12
8. Export des résultats en CSV et récupération sur la machine locale.....	14
8.1 Organisation des exports dans HDFS	14
Vérifier l'existence des fichiers :.....	14
8.2 Copier depuis HDFS vers le PC.....	14

9. Visualisation dans Power BI Desktop 16

9.1 Import des fichiers CSV 16

9.2 Visualisations analytiques réalisées 16

1. Introduction

Ce rapport présente la conception et la mise en œuvre d'un pipeline Big Data complet répondant à la chaîne classique ingestion → stockage → traitement → visualisation. Le cas d'usage choisi est l'analyse de logs applicatifs (microservices), un besoin très répandu dans les environnements cloud et DevOps (observabilité, SRE, supervision, détection d'incidents).

Le pipeline mis en place s'appuie sur des composants standards, déployés via Docker Compose : Apache Kafka pour l'ingestion, Apache Spark pour le traitement (streaming et batch), HDFS pour le stockage, puis une étape d'export en CSV afin d'alimenter Power BI Desktop pour la visualisation.

Objectifs pédagogiques et techniques :

- Concevoir un pipeline end-to-end (de la donnée brute à un tableau de bord).
- Manipuler le traitement en temps réel (Spark Structured Streaming).
- Réaliser des analyses batch via Spark SQL.
- Stocker et organiser des données dans HDFS.
- Exporter les résultats et les consommer dans un outil BI (Power BI).
- Documenter l'architecture, les choix techniques, et les étapes d'exécution.

Périmètre du travail :

Le pipeline couvre :

- (1) génération ou simulation de logs
- (2) publication dans Kafka (topic logs)
- (3) consommation et enrichissement en streaming par Spark
- (4) écriture des logs normalisés dans HDFS
- (5) exécution d'analyses batch (agrégations)
- (6) export de jeux de données en CSV
- (7) création de visuels dans Power BI

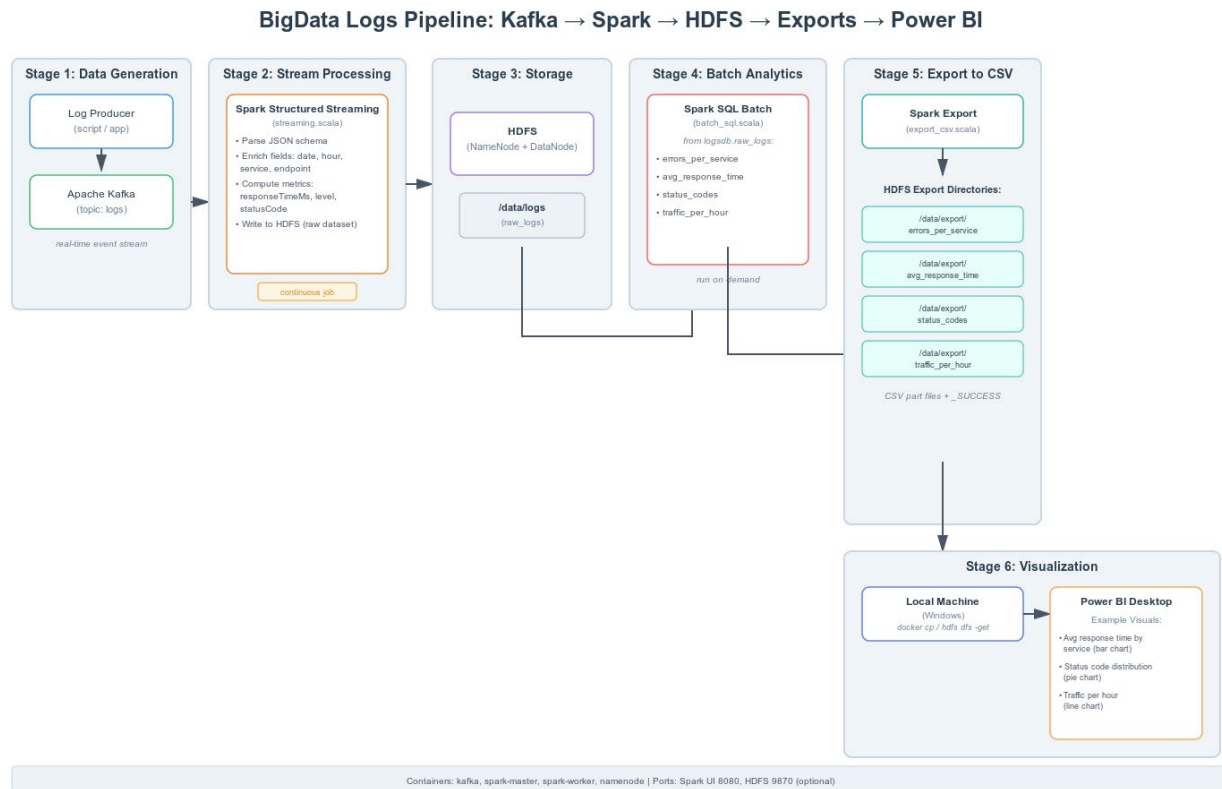


Figure 1 : Schéma global du pipeline Big Data de traitement des logs (Kafka → Spark → HDFS → Export CSV → Power BI)

2. Contexte et cas d'usage (logs applicatifs)

Dans un système applicatif moderne, souvent basé sur une architecture **microservices**, chaque service applicatif produit en continu des logs techniques. Ces logs contiennent des informations essentielles telles que les traces d'accès, les erreurs, les temps de réponse, les codes HTTP et divers messages applicatifs.

Ces données sont indispensables pour :

- diagnostiquer rapidement des pannes (pics d'erreurs, timeouts),
- mesurer les performances applicatives (latence moyenne par endpoint),
- suivre la charge du système (volume de requêtes par heure),
- détecter des comportements anormaux ou des anomalies (variations brusques, erreurs répétées).

Dans le cadre de ce projet académique, les logs ne proviennent pas d'un système de production réel, mais sont **générés de manière simulée** à l'aide d'un script applicatif. Cette approche permet de :

- reproduire un volume important de données,

- simuler différents scénarios réalistes (succès, erreurs, latence élevée),
- tester le pipeline Big Data dans des conditions proches du réel,
- se concentrer sur la conception et l'analyse de l'architecture distribuée.

Les logs sont générés sous forme **structurée (format JSON)**, puis envoyés en temps réel vers un système de streaming. Ils sont ensuite enrichis, stockés et analysés afin de produire des **indicateurs de performance (KPI)** exploitables dans un tableau de bord décisionnel.

Exemple de structure d'un log (JSON) :

```
{
  "event_time": "2025-12-24T18:00:10Z",
  "service": "orders",
  "level": "ERROR",
  "endpoint": "/api/orders/123",
  "statusCode": 500,
  "responseTimeMs": 850,
  "host": "orders-1",
  "message": "DB timeout"
}
```

Chaque champ a un rôle : event_time (horodatage), service (microservice), level (niveau), endpoint (API appelée), statusCode (code HTTP), responseTimeMs (latence), host (instance), message (détail technique).

```
{
  "ts": "2025-12-24T18:42:02.187Z",
  "host": "server-2",
  "service": "auth",
  "level": "ERROR",
  "endpoint": "/search",
  "statusCode": 504,
  "responseTimeMs": 3230,
  "message": "Something failed"
}
{"ts": "2025-12-24T18:42:02.187Z", "host": "server-2", "service": "auth", "level": "INFO", "endpoint": "/login", "statusCode": 201, "responseTimeMs": 807, "message": "OK"}
{"ts": "2025-12-24T18:42:02.187Z", "host": "server-3", "service": "search", "level": "ERROR", "endpoint": "/checkout", "statusCode": 502, "responseTimeMs": 3099, "message": "Something failed"}
{"ts": "2025-12-24T18:42:02.187Z", "host": "server-3", "service": "search", "level": "INFO", "endpoint": "/pay", "statusCode": 204, "responseTimeMs": 92, "message": "OK"}
{"ts": "2025-12-24T18:42:02.187Z", "host": "server-3", "service": "orders", "level": "INFO", "endpoint": "/login", "statusCode": 204, "responseTimeMs": 307, "message": "OK"}
{"ts": "2025-12-24T18:42:02.187Z", "host": "server-1", "service": "payments", "level": "WARN", "endpoint": "/checkout", "statusCode": 403, "responseTimeMs": 901, "message": "Suspicious / slow request"}
}
{"ts": "2025-12-24T18:42:02.187Z", "host": "server-3", "service": "auth", "level": "WARN", "endpoint": "/search", "statusCode": 401, "responseTimeMs": 1079, "message": "Suspicious / slow request"}
{"ts": "2025-12-24T18:42:02.187Z", "host": "server-3", "service": "auth", "level": "INFO", "endpoint": "/admin", "statusCode": 204, "responseTimeMs": 1272, "message": "OK"}
{"ts": "2025-12-24T18:42:02.187Z", "host": "server-1", "service": "gateway", "level": "WARN", "endpoint": "/search", "statusCode": 401, "responseTimeMs": 1770, "message": "Suspicious / slow request"}
{"ts": "2025-12-24T18:42:02.187Z", "host": "server-2", "service": "gateway", "level": "INFO", "endpoint": "/checkout", "statusCode": 201, "responseTimeMs": 447, "message": "OK"}
{"ts": "2025-12-24T18:42:02.187Z", "host": "server-1", "service": "auth", "level": "INFO", "endpoint": "/login", "statusCode": 204, "responseTimeMs": 488, "message": "OK"}
{"ts": "2025-12-24T18:42:02.187Z", "host": "server-3", "service": "orders", "level": "WARN", "endpoint": "/pay", "statusCode": 401, "responseTimeMs": 644, "message": "Suspicious / slow request"}
{"ts": "2025-12-24T18:42:02.187Z", "host": "server-2", "service": "orders", "level": "INFO", "endpoint": "/login", "statusCode": 204, "responseTimeMs": 1110, "message": "OK"}
{"ts": "2025-12-24T18:42:03.198Z", "host": "server-1", "service": "payments", "level": "INFO", "endpoint": "/pay", "statusCode": 200, "responseTimeMs": 1468, "message": "OK"}
{"ts": "2025-12-24T18:42:03.198Z", "host": "server-2", "service": "catalog", "level": "INFO", "endpoint": "/checkout", "statusCode": 200, "responseTimeMs": 1011, "message": "OK"}
{"ts": "2025-12-24T18:42:03.198Z", "host": "server-1", "service": "search", "level": "INFO", "endpoint": "/checkout", "statusCode": 201, "responseTimeMs": 207, "message": "OK"}
{"ts": "2025-12-24T18:42:03.198Z", "host": "server-3", "service": "orders", "level": "INFO", "endpoint": "/search", "statusCode": 201, "responseTimeMs": 71, "message": "OK"}
{"ts": "2025-12-24T18:42:03.198Z", "host": "server-1", "service": "auth", "level": "INFO", "endpoint": "/checkout", "statusCode": 201, "responseTimeMs": 271, "message": "OK"}
{"ts": "2025-12-24T18:42:03.198Z", "host": "server-1", "service": "catalog", "level": "WARN", "endpoint": "/login", "statusCode": 400, "responseTimeMs": 2069, "message": "Suspicious / slow request"}
{"ts": "2025-12-24T18:42:03.198Z", "host": "server-1", "service": "orders", "level": "INFO", "endpoint": "/checkout", "statusCode": 201, "responseTimeMs": 830, "message": "OK"}
{"ts": "2025-12-24T18:42:03.198Z", "host": "server-2", "service": "orders", "level": "INFO", "endpoint": "/products", "statusCode": 201, "responseTimeMs": 777, "message": "OK"}
}
```

Figure 2 : Exemple de logs applicatifs structurés générés au format JSON

3. Architecture générale et composants

L'architecture est déployée sur une seule machine via Docker Compose. Chaque composant s'exécute dans un conteneur dédié. Cette approche facilite la reproductibilité et l'isolation des dépendances.

3.1 Vue d'ensemble des étapes

1. Étape 1 - Génération de données : script Node.js (producer.js).
2. Étape 2 - Ingestion : publication dans Kafka (topic logs).
3. Étape 3 - Traitement streaming : Spark Structured Streaming consomme Kafka, enrichit, écrit dans HDFS.
4. Étape 4 - Stockage : HDFS conserve les logs bruts/normalisés.
5. Étape 5 - Traitement batch : Spark SQL calcule des agrégations (KPI).
6. Étape 6 - Export : écriture des KPI en CSV (un dossier par indicateur).
7. Étape 7 - Visualisation : import des CSV dans Power BI + création de visuels et filtres.

3.2 Conteneurs et rôles

Conteneur	Rôle	Ports exposés
kafka	Broker Kafka + contrôleur (KRaft), ingestion via topic logs	9092
namenode	HDFS NameNode (métadonnées, namespace)	9870
datanode	HDFS DataNode (blocs de données)	9864
spark-master	Spark Master (gestion cluster)	8080, 7077
spark-worker	Spark Worker (exécuteurs)	8081

Remarque : Le port 8080 permet d'accéder à l'interface web du Spark Master (applications, workers, ressources). Le port 9870 expose l'interface HDFS du NameNode (exploration des répertoires, statut du cluster).

[Give feedback](#)

[Learn more](#)

Data unavailable at this time

Container memory usage

Data unavailable at

Q Search

☐

Only show running containers

		Name	Container ID	Image	Port(s)
☒	▼	 bigdata-logs-pipeline	-	-	-
☒		 spark-worker-1	b0af7b9ff23f	bde2020/s	8081:8081 ↗
☒		 spark-master-1	dceaca6868c6	bde2020/s	7077:7077 ↗ Show all ports (2)
☒		 datanode-1	230605fd4325	bde2020/h	9864:9864 ↗
☒		 namenode-1	c1027f3e660f	bde2020/h	9870:9870 ↗
☒		 kafka-1	d2c5b5c11e8f	apache/kaf	9092:9092 ↗

Figure 3 : liste des conteneurs (docker desktop)

spark 3.1.1

Spark Master at spark://dceaca6868c6:7077

URL: spark://dceaca6868c6:7077

Alive Workers: 1

Cores in use: 8 Total, 8 Used

Memory in use: 2.7 GiB Total, 1024.0 MiB Used

Resources in use:

Applications: 1 Running, 1 Completed

Drivers: 0 Running, 0 Completed

Status: ALIVE

Workers (1)

Worker Id	Address	State	Cores	Memory	Resources
worker-20251224154855-172.19.0.6-40111	172.19.0.6:40111	ALIVE	8 (8 Used)	2.7 GiB (1024.0 MiB Used)	

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20251224155753-0001	(kill) Spark shell	8	1024.0 MiB		2025/12/24 15:57:53	root	RUNNING	1.1 min

Completed Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20251224155109-0000	Spark shell	8	1024.0 MiB		2025/12/24 15:51:09	root	FINISHED	2.4 min

Figure 4: Spark Master UI (localhost:8080) montrant worker(s) et applications

4. Déploiement avec Docker Compose

Cette section décrit le fichier docker-compose.yml et les paramètres clés utilisés pour faire fonctionner Kafka, HDFS et Spark dans un réseau Docker commun.

4.1 Configuration Kafka

Kafka est déployé avec l'image apache/kafka:3.7.0. La configuration utilise trois listeners (CONTROLLER, HOST, DOCKER) afin de permettre :

- la communication interne Kafka (mode KRaft) via CONTROLLER.
- l'accès depuis la machine hôte (HOST://localhost:9092).
- la communication entre conteneurs (DOCKER://kafka:9093).

4.2 Configuration HDFS (NameNode/DataNode)

HDFS est assuré par les images bde2020/hadoop-namenode et bde2020/hadoop-datanode. Le NameNode conserve les métadonnées du système de fichiers (arborescence, permissions, localisation des blocs). Le DataNode stocke les blocs de fichiers. Des volumes Docker (namenode, datanode) assurent la persistance.

4.3 Configuration Spark (master/worker)

Spark est déployé en mode cluster standalone : un master (spark://spark-master:7077) et un worker enregistré sur ce master. Les scripts Scala sont montés dans /opt/spark-apps via un volume bind (./spark).

5. Ingestion : génération de logs et publication dans Kafka

L'ingestion est assurée par un producteur (producer.js) qui simule des logs applicatifs et les publie dans Kafka (topic logs). Cette simulation permet de tester le pipeline sans dépendre d'une application réelle.

5.1 Producteur Node.js (producer.js)

Le producteur génère périodiquement des événements JSON. Chaque événement est sérialisé en texte et envoyé vers Kafka. Un point important est de produire des données suffisamment variées (services, endpoints, niveaux, codes) pour rendre les analyses intéressantes.

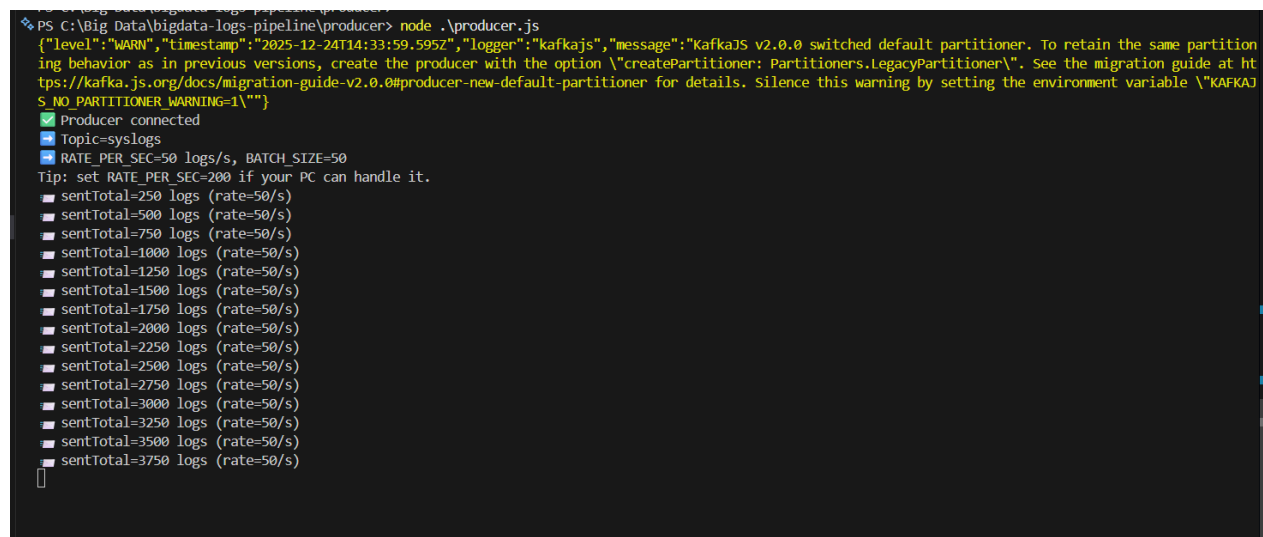
Exemples de variations utiles :

- levels : INFO, WARN, ERROR.
- statusCode : 200, 201, 400, 401, 403, 404, 429, 500, 502, 503, 504.
- responseTimeMs : faible (50-200 ms) jusqu'à élevé (> 1000 ms).
- service : auth, payments, orders, catalog, gateway, search.

Commandes (exemple) :

```
# (Optionnel) créer le topic logs
docker exec -it bigdata-logs-pipeline-kafka-1 bash -lc "kafka-topics.sh --bootstrap-server kafka:9093 --create --topic logs --partitions 1 --replication-factor 1"

# Lancer le producteur (dans un conteneur dédié ou depuis l'hôte)
node producer/producer.js
```



```
PS C:\Big Data\bigdata-logs-pipeline\producer> node .\producer.js
{"level":"WARN","timestamp":"2025-12-24T14:33:59.595Z","logger":"kafka.js","message":"KafkaJS v2.0.0 switched default partitioner. To retain the same partitioning behavior as in previous versions, create the producer with the option \\'createPartitioner: Partitioners.LegacyPartitioner\\'. See the migration guide at https://kafka.js.org/docs/migration-guide-v2.0.0#producer-new-default-partitioner for details. Silence this warning by setting the environment variable \\'KAFKAJS_NO_PARTITIONER_WARNING=1\\'"}
S_NO_PARTITIONER_WARNING=1\''}
✓ Producer connected
Topic=syslogs
RATE_PER_SEC=50 logs/s, BATCH_SIZE=50
Tip: set RATE_PER_SEC=200 if your PC can handle it.
sentTotal=250 logs (rate=50/s)
sentTotal=500 logs (rate=50/s)
sentTotal=750 logs (rate=50/s)
sentTotal=1000 logs (rate=50/s)
sentTotal=1250 logs (rate=50/s)
sentTotal=1500 logs (rate=50/s)
sentTotal=1750 logs (rate=50/s)
sentTotal=2000 logs (rate=50/s)
sentTotal=2250 logs (rate=50/s)
sentTotal=2500 logs (rate=50/s)
sentTotal=2750 logs (rate=50/s)
sentTotal=3000 logs (rate=50/s)
sentTotal=3250 logs (rate=50/s)
sentTotal=3500 logs (rate=50/s)
sentTotal=3750 logs (rate=50/s)
```

Figure 5 : exécution du producer (logs dans la console).

6. Traitement temps réel : Spark Structured Streaming

Spark Structured Streaming consomme les messages Kafka en continu. L'objectif est de : (1) parser le JSON, (2) normaliser les types, (3) enrichir la donnée (date, heure), (4) écrire en continu dans HDFS.

6.1 Lecture depuis Kafka

Spark se connecte au broker Kafka via `bootstrap.servers`. Les messages sont lus comme des lignes (value) qu'on convertit ensuite en chaîne JSON puis en colonnes.

6.2 Transformation et enrichissement

Après parsing, on ajoute typiquement :

- une colonne timestamp exploitable
- des colonnes date et hour (pour l'agrégation)
- éventuellement des tags (ex : request slow, big move, etc.)

6.3 Écriture dans HDFS

Les logs normalisés sont écrits en mode append dans HDFS, dans un répertoire dédié. On peut partitionner par date/heure pour améliorer la performance de lecture ultérieure.

Exemples de chemins HDFS :

- `/data/logs/raw_logs/` (données normalisées par Spark).
- `/data/export/...` (résultats agrégés exportés).

Démarrage du streaming (exécution interactive) :

```
docker exec -it bigdata-logs-pipeline-spark-master-1 bash -lc "  
/spark/bin/spark-shell --master spark://spark-master:7077 --conf  
spark.hadoop.fs.defaultFS=hdfs://bigdata-logs-pipeline-namenode-1:8020 -i /opt/spark-  
apps/streaming.scala  
"
```

L'application reste active car c'est du streaming (continu).

Important : un job streaming ne se termine pas automatiquement. Il est normal de le laisser tourner pendant la phase de génération de données. On l'arrête seulement lorsque l'on veut figer un jeu de données pour lancer des analyses batch.

Arrêt propre :

Selon le code, l'arrêt peut se faire :

- via Ctrl+C si on est dans le spark-shell
- via l'UI Spark Master (bouton kill sur l'application)
- ou en terminant le conteneur (à éviter si possible).

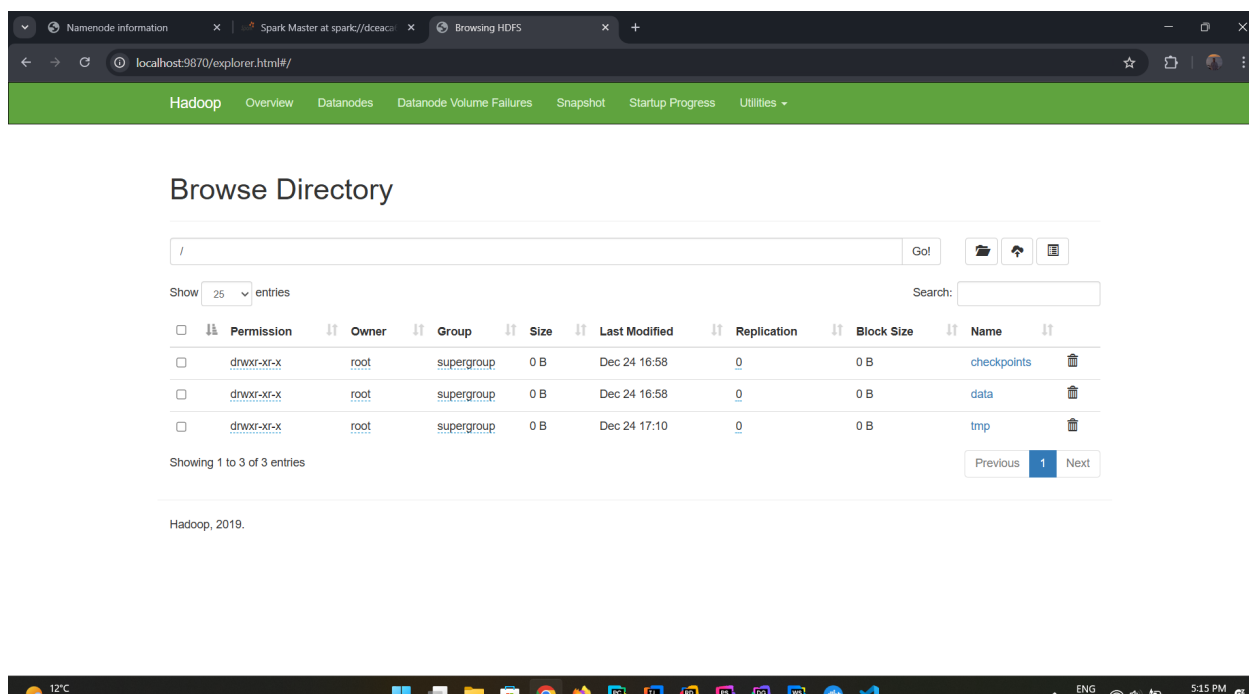


Figure 6 :HDFS UI (9870) montrant l'écriture des fichiers

7. Analytique batch : Spark SQL

Une fois un volume de logs suffisant généré et stocké dans HDFS, on exécute des requêtes batch pour calculer des indicateurs agrégés. Cette étape est adaptée aux rapports périodiques (par heure, par jour) et aux tableaux de bord.

7.1 Mise en place de la base logique (logsdb)

Dans le script `batch_sql.scala`, on lit les logs depuis HDFS, on infère ou impose un schéma, puis on crée une base et une table (ou une vue) afin d'utiliser Spark SQL.

7.2 Indicateurs calculés

Les KPI retenus pour le projet :

- Nombre d'erreurs par service (ERROR count).
- Temps de réponse moyen par endpoint (AVG responseTimeMs).
- Répartition des statusCode (COUNT par code).
- Trafic par heure (COUNT par date, hour).

Ces indicateurs sont suffisamment génériques pour un scénario réel : identifier les services les plus instables, repérer des endpoints lents, et observer la charge dans le temps.

Exécution du batch :

```
docker exec -it bigdata-logs-pipeline-spark-master-1 bash -lc "
/spark/bin/spark-shell --master spark://spark-master:7077 --conf
spark.hadoop.fs.defaultFS=hdfs://bigdata-logs-pipeline-namenode-1:8020 --conf
```

```
spark.sql.shuffle.partitions=4 --conf spark.driver.memory=1g --conf
spark.executor.memory=1g -i /opt/spark-apps/batch_sql.scala
"
# Le batch doit se terminer (FINISHED) après exécution des requêtes.
```

Si le job reste en état WAITING, cela indique généralement que les ressources ne sont pas allouées (worker non enregistré, mémoire insuffisante) ou qu'un autre job monopolise les coeurs. Dans ce cas, vérifier l'UI Spark Master et arrêter les applications inutiles.

Résultats Spark SQL, erreurs par service et temps de réponse moyen par endpoint

ts	service	level	endpoint	statusCode	responseTime	host
2025-12-24T16:09:15.133Z	orders	INFO	/products	201	1038	server-3
2025-12-24T16:09:15.133Z	orders	INFO	/admin	200	301	server-3
2025-12-24T16:09:15.133Z	orders	INFO	/login	201	1260	server-1
2025-12-24T16:09:15.133Z	catalog	INFO	/admin	204	475	server-3
2025-12-24T16:09:15.133Z	catalog	INFO	/checkout	201	1155	server-1
2025-12-24T16:09:15.133Z	search	INFO	/admin	200	559	server-2
2025-12-24T16:09:15.133Z	orders	WARN	/products	400	2340	server-2
2025-12-24T16:09:15.133Z	gateway	INFO	/checkout	204	320	server-3
2025-12-24T16:09:15.133Z	search	ERROR	/login	500	3067	server-2
2025-12-24T16:09:15.133Z	catalog	INFO	/search	200	632	server-3
2025-12-24T16:09:15.133Z	search	INFO	/search	201	1492	server-2
2025-12-24T16:09:15.133Z	payments	INFO	/search	204	428	server-2
2025-12-24T16:09:15.133Z	catalog	INFO	/products	200	430	server-3
2025-12-24T16:09:15.133Z	search	INFO	/login	204	163	server-2
2025-12-24T16:09:15.133Z	payments	INFO	/search	201	546	server-3
2025-12-24T16:09:15.133Z	gateway	ERROR	/pay	503	4042	server-2
2025-12-24T16:09:15.133Z	search	WARN	/products	429	1425	server-3
2025-12-24T16:09:15.133Z	payments	INFO	/pay	204	861	server-1
2025-12-24T16:09:15.133Z	gateway	INFO	/pay	200	982	server-1
2025-12-24T16:09:15.133Z	auth	INFO	/admin	200	395	server-3

statusCode	cnt
204	71570
200	71524
201	70981
401	10045
429	10020
403	9972
400	9867
502	3539
503	3369
500	3357
504	3356

date	hour	total_logs
2025-12-24	16	27550
2025-12-24	15	162650
2025-12-24	14	77400

service	error_count
catalog	2327
payments	2299
gateway	2295
search	2262
auth	2240
orders	2198

endpoint	avg_rt	n
/admin	958.9788420460933	44475
/products	957.9748844685554	44793
/checkout	957.2002378441448	44567
/pay	953.7580420522958	44516
/search	953.6147845967633	44614
/login	953.5857510921923	44635

8. Export des résultats en CSV et récupération sur la machine locale

Après calcul des KPI, l'objectif est de produire des fichiers CSV faciles à importer dans Power BI. Chaque indicateur est écrit dans un répertoire distinct. Spark écrit par défaut plusieurs 'part-*' fichiers, mais l'utilisation de coalesce(1) permet d'obtenir un seul fichier de données par export (au prix d'un regroupement sur une seule partition).

8.1 Organisation des exports dans HDFS

Indicateur	Chemin HDFS	Champs attendus
Erreurs par service	/data/export/errors_per_service	service, error_count
Temps de réponse moyen	/data/export/avg_response_time	endpoint, avg_rt
Répartition status codes	/data/export/status_codes	statusCode, cnt
Trafic par heure	/data/export/traffic_per_hour	date, hour, total_logs

Vérifier l'existence des fichiers :

```
docker exec -it bigdata-logs-pipeline-namenode-1 hdfs dfs -ls /data/export/errors_per_service
# On doit voir _SUCCESS et part-00000-....csv
```

8.2 Copier depuis HDFS vers le PC

Méthode recommandée (en deux étapes) :

- 1) Copier les répertoires HDFS vers un dossier local dans le conteneur namenode (hdfs dfs -get)
- 2) Copier ce dossier du conteneur vers Windows avec docker cp

Attention : dans certains cas, la commande 'hdfs' n'est pas disponible dans un shell bash si l'environnement n'est pas initialisé. Utiliser directement 'hdfs dfs ...' dans docker exec comme ci-dessous.

```
# 1) Créer un dossier temporaire dans le conteneur NameNode
docker exec -it bigdata-logs-pipeline-namenode-1 bash -lc "mkdir -p /tmp/export"

# 2) Copier depuis HDFS vers /tmp/export (dans le conteneur)
docker exec -it bigdata-logs-pipeline-namenode-1 hdfs dfs -get /data/export /tmp/export

# 3) Copier du conteneur vers votre PC (Windows)
docker cp bigdata-logs-pipeline-namenode-1:/tmp/export ./export_csv
```

À la fin, sur votre PC, vous obtenez un dossier export_csv contenant un sous-dossier par indicateur, et à l'intérieur le fichier part-00000-...csv (ainsi que _SUCCESS).

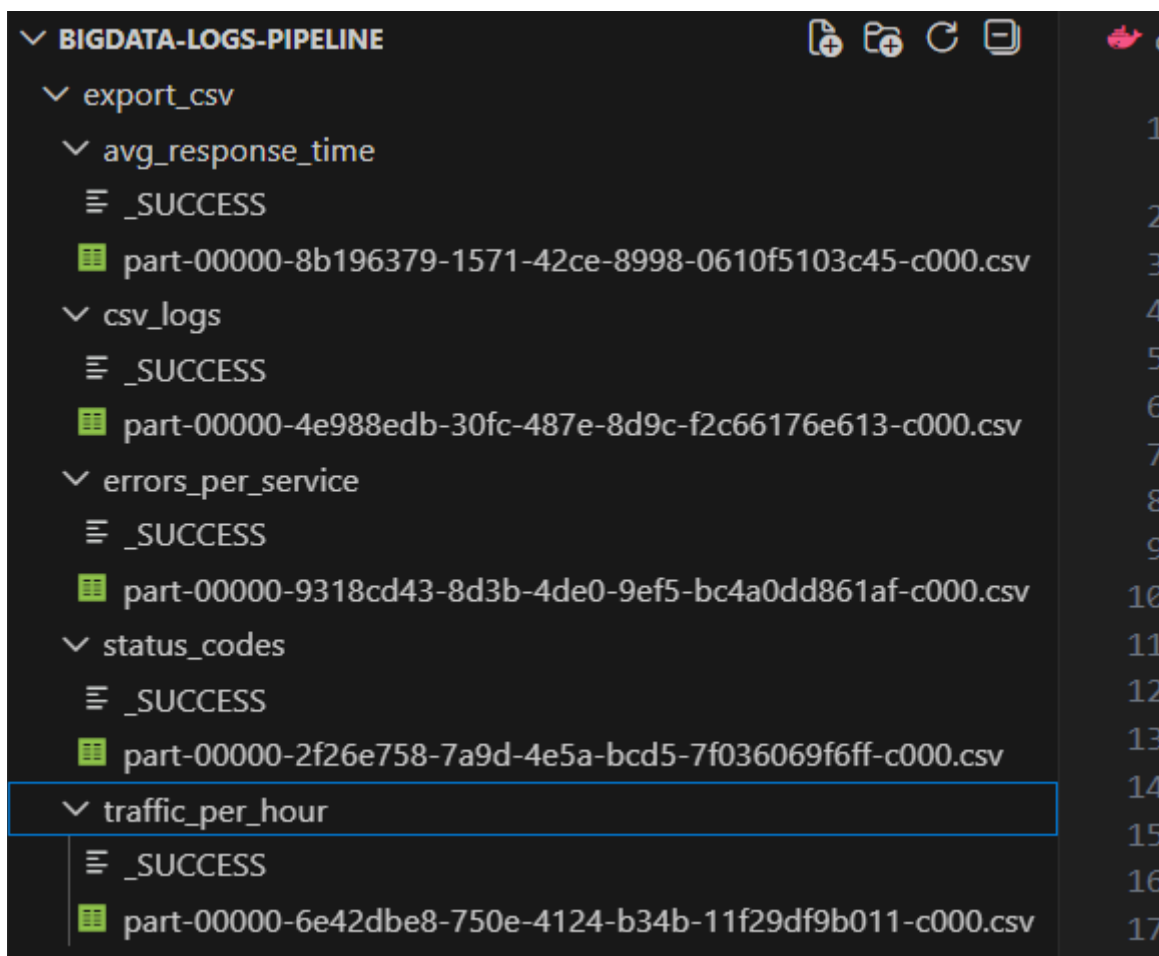


Figure 11 : Arborescence export_csv sur Windows

9. Visualisation dans Power BI Desktop

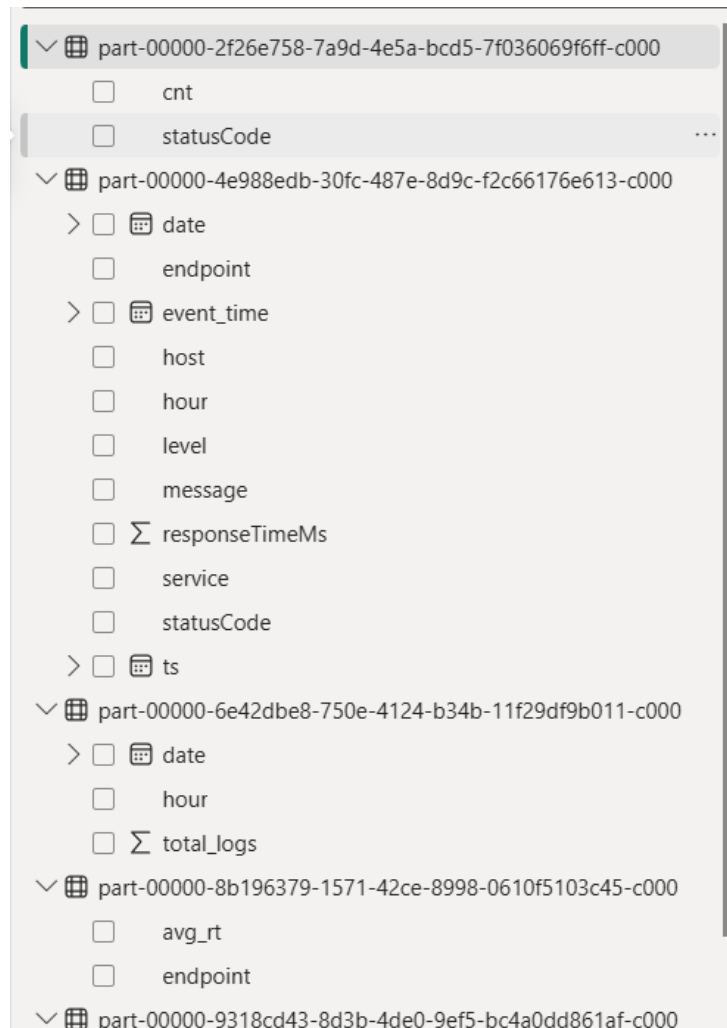
Power BI est utilisé pour construire un tableau de bord à partir des CSV exportés. Chaque CSV correspond à une table. Dans un contexte réel, on peut aussi créer un modèle relationnel (relations entre tables) mais ici les indicateurs sont déjà agrégés, donc on peut visualiser directement.

9.1 Import des fichiers CSV

Dans Power BI Desktop :

Accueil → Obtenir des données → Texte/CSV → sélectionner le fichier part-00000-...csv.

Répéter l'opération pour chaque indicateur (errors_per_service, avg_response_time, status_codes, traffic_per_hour).



9.2 Visualisations analytiques réalisées

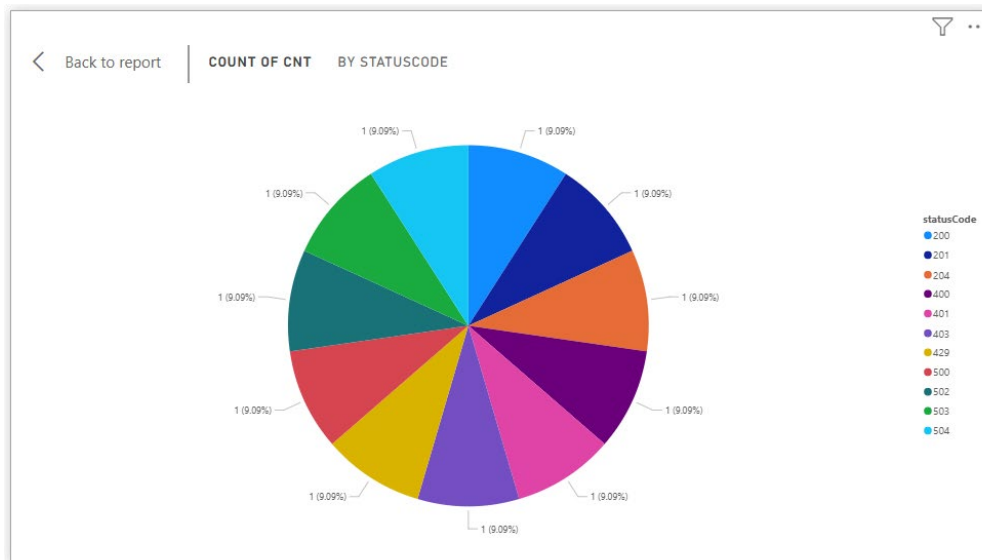


Figure 13 :Résultats Spark SQL – Nombre d’erreurs (level=ERROR) par service et latence moyenne par endpoint

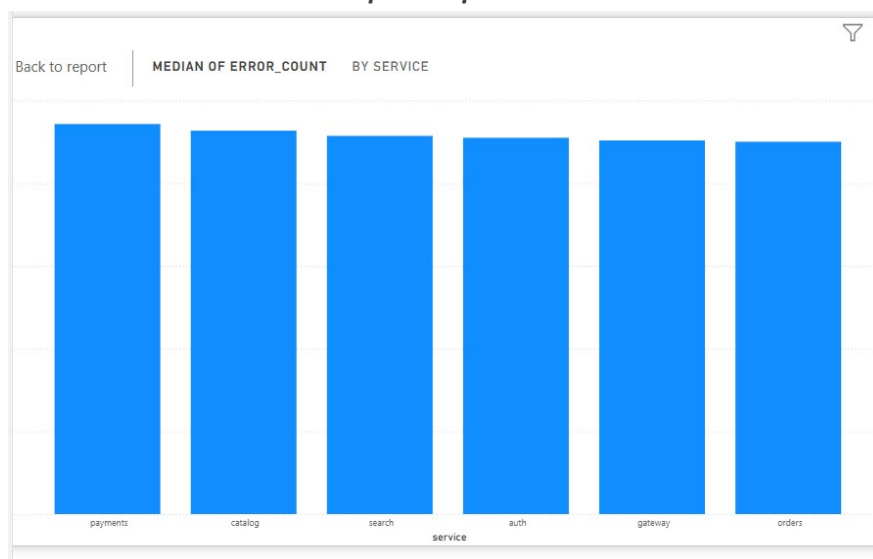


Figure 14 : Résultats Spark SQL – Répartition des codes HTTP (statusCode) et volume de logs par heure

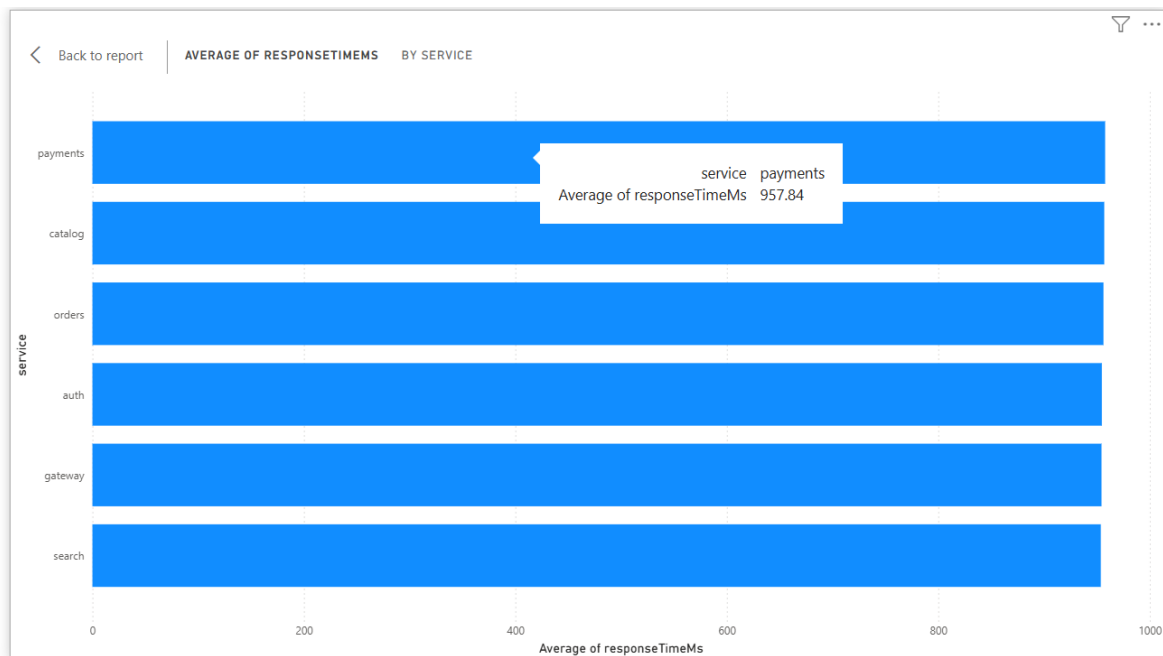


Figure 15 : Aperçu des logs enrichis (DataFrame) après traitement Spark (service, level, endpoint, statusCode, responseTimeMs, host)